

LISTA DE EXERCÍCIOS 1.3

1. Quais são as principais diferenças entre sistemas operacionais para *mainframes* e computadores pessoais?

Resposta: Geralmente, SOs para sistemas em lote possuem requisitos mais simples do que no caso de computadores pessoais. Sistemas em lote não precisam se preocupar com a interação com um usuário assim como acontece em um computador pessoal. Como resultado, um SO para um PC deve lidar com tempos de resposta para um usuário interativo. Sistemas em lote não possuem tais requisitos. Um sistema em lote puro, também pode não precisar tratar a questão de tempo compartilhado (*time sharing*), enquanto um SO para PC deve chavear rapidamente entre diferentes tarefas (*jobs*).

2. Como a distinção entre modo de kernel e modo de usuário funcionam como uma forma de sistema de proteção (segurança)?

Resposta: A distinção entre modo de kernel e modo de usuário oferece uma forma rudimentar de proteção considerando que certas instruções poderiam ser executadas apenas quando a CPU está em modo de kernel. De maneira similar, dispositivos de hardware poderiam ser acessados somente quando o programa está executando em modo de kernel. O controle sobre o momento de ocorrência de uma interrupção poderia ser habilitado ou desabilitado apenas quando a CPU também está executando em modo de kernel. Dessa forma, a CPU possui uma capacidade muito limitada ao executar em modo de usuário, reforçando a proteção de recursos críticos.

3. Qual das instruções abaixo relacionadas devem ser privilegiadas (modo de kernel)?
 - a) Ajuste de *timer*
 - b) Leitura do *clock*
 - c) Limpeza de memória (de sistema)
 - d) Emitir uma *trap*
 - e) Desabilitar as interrupções
 - f) Modificar as entradas na tabela de *status* de dispositivo
 - g) Chavear de modo usuário para modo kernel
 - h) Acessar dispositivo de E/S

Resposta: As seguintes operações precisam ser privilegiadas: ajuste do valor da **interrupção de *timer***¹ (existem timers que podem ser ajustados pelo usuário – vide exemplo da questão 8), limpeza de memória (pelo SO), desabilitar as interrupções, modificar as entradas na tabela de *status* de dispositivo, acesso a dispositivo de E/S (pós execução da *syscall*). O restante pode ser realizado em espaço de usuário.

4. Algumas CPUs oferecem mais do que dois modos de operação. Quais são dois possíveis usos destes múltiplos modos?

Resposta: Embora a maioria dos sistemas façam distinção apenas entre os modos de usuário e kernel, algumas CPUs possuem suporte a múltiplos modos. Múltiplos modos poderiam ser usados para oferecer uma política de segurança mais detalhada. Por exemplo, ao invés de distinguir apenas entre os modos de usuário e kernel, haveria a possibilidade de distinguir entre diferentes tipos de modo de usuário. Talvez, usuários pertencentes ao mesmo grupo pudessem executar o código uns dos outros.

¹http://en.wikibooks.org/wiki/Windows_Programming/User_Mode_vs_Kernel_Mode

A máquina entraria em um modo especificado, quando um destes usuários estivesse executando um dado código. Quando a máquina estivesse neste modo, um membro do grupo poderia executar o código pertencente a qualquer membro do mesmo grupo. Outra possibilidade, seria oferecer distinções diferentes entre os códigos de kernel. Por exemplo, um modo específico poderia permitir que dispositivos USB executassem. Isso significaria que, dispositivos USB poderiam ser servidos sem chavear para o modo kernel, permitindo de certa forma, que estes executassem em modos usuário/kernel próprios.

~~5. Interrupções de *timer* poderiam ser usadas para computar o tempo corrente. Ofereça uma descrição de como isto poderia ser alcançado?~~

6. Explique o princípio de multiprogramação e seu papel quanto a eficiência do sistema.

Resposta: Multiprogramação é necessária para garantir a eficiência por meio do balanceamento da utilização de recursos do sistema. Dessa forma, um único usuário não pode manter a CPU e dispositivos de E/S ocupados todo o tempo. Com o artifício da multiprogramação, é possível organizar *jobs* (código e dados) de forma que a CPU sempre tenha um *job* para ser executado. Um subconjunto do total de jobs no sistema é mantido na memória. Com base em algum algoritmo de escalonamento, um job é selecionado e executa via escalonamento de job. Quando ele deve esperar (E/S por exemplo), o SO executa outro *job*.